

ON-THE-METAL SOVEREIGN AI

One binary. Zero egress. On the metal.

A **54 MB statically-linked Rust binary** carries the entire AI control plane and execution layer — gateway, four-stage governance, MCP host, scheduler, audit-spine — inside a single process. Drop it onto a sealed network, point it at an on-prem inferencer (vLLM, Llama, Qwen — any / v1/messages), and the AI stack runs with **no outbound calls and no host-routable interface** on the engine itself. Built for sovereign, defence, and hardware-integration deployments where the wire is the boundary: copy the binary, supply secrets, run.

AT A GLANCE

One binary. On the metal.

54 MB

SINGLE RUST BINARY ·
CONTROL + EXECUTION

0

OUTBOUND PACKETS ·
SIDECAR-ASSERTED

1

HOST-ROUTABLE PORT ·
SOCAT DEMARCATION

14ms

GOVERNANCE P95 ·
SEALED RUN

01 Core capabilities

>

BINARY

One binary, on the metal

54 MB statically-linked Rust binary. Control plane and execution path live in **one process**. No interpreter, no cloud agent. Copy, run, done.

54 MB

>

SEAL

Sealed Docker network

internal: true on the app network. No gateway, no NAT, no host-routable path. The app container itself never joins a bridge.

Zero egress

>

SANDBOX

Sandboxed control + execution

Gateway, 4-stage governance, MCP host, in-network inferencer — all behind the air gap. Un-listed models denied **403 pre-connect**; secret-scan, blocklist, rate-limit in-process.

policy 403

>

INSTALL

Hardware-ready install

Profile dir mounts :ro. No boot-time writes, no internet bootstrap. Pre-supplied signing seed, stable identity across power-cycles. Suitable for hardware and disconnected sites.

Build · ship · run

02 Why sovereign deployments choose systemprompt.io

→ **Compiled to a single binary you can ship on a USB stick**

54 MB statically-linked Rust binary — gateway, governance, MCP host, scheduler, audit-spine writers, all inside one process. No interpreter, no sidecar agent, no cloud bootstrap; `copy + run` is the entire install.

→ **Control plane and execution co-located behind the air gap**

The `app` is the AI control plane and the path to the on-prem inferencer (vLLM / Llama / Qwen / any /v1/messages). Gateway, governance, audit, provider call — all inside the sealed network. No second hop to SaaS.

→ **Built for hardware and disconnected sites**

Read-only profile mount, no boot-time writes, no start-up outbound calls. Allow-list ships as version-controlled YAML (`services/ai/gateway-policies.yaml`), ingested at every boot with `delete_orphans: true` — re-buildable from source for ITAR / IL-pattern sites.

TOPOLOGY	Two networks: <code>airgap-internal</code> (<code>internal: true</code>) and a tiny bridge for the <code>ingress</code> <code>socat</code> . The <code>socat</code> is the only container on both.
CLOSURE PROOFS	(1) <code>ss -tunp + conntrack -L</code> from a monitor sidecar; (2) in-network responder <code>/stats</code> unchanged on a denied-burst run. Both run by <code>01-egress-assert.sh</code> .
IDENTITY	JWT-bound. RBAC + 4-stage governance pipeline (<code>scope · secret-scan · blocklist · rate-limit</code>) on every <code>/v1/messages</code> call.
DEPLOYMENT	Build → ship → install. <code>:ro</code> profile dir, no outbound calls required at boot. Pre-supplied signing seed for stable identity across power-cycles. Suitable for hardware integrations and disconnected sites.
OBSERVABILITY	Audit-spine in your Postgres. Optional OpenTelemetry exporter; no required outbound telemetry path.
COMPLIANCE	SOC 2 (CC6.1, CC7.2), ISO 27001 (A.5.15, A.8.3, A.8.25), HIPAA §164.312, EU AI Act Art. 9/12/15, NIST AI RMF, OWASP Agentic Top 10. ITAR / IL-pattern sovereign deployments supported.

IN-NETWORK INFERENCE · SWAPPABLE

Demo: deterministic mock-inference (Anthropic Messages-compatible) **Production:** on-prem vLLM · Llama · Qwen · any /v1/messages

Endpoint resolution: `${ANTHROPIC_ENDPOINT}` → in-network URL; provider stays enabled even without an upstream key.

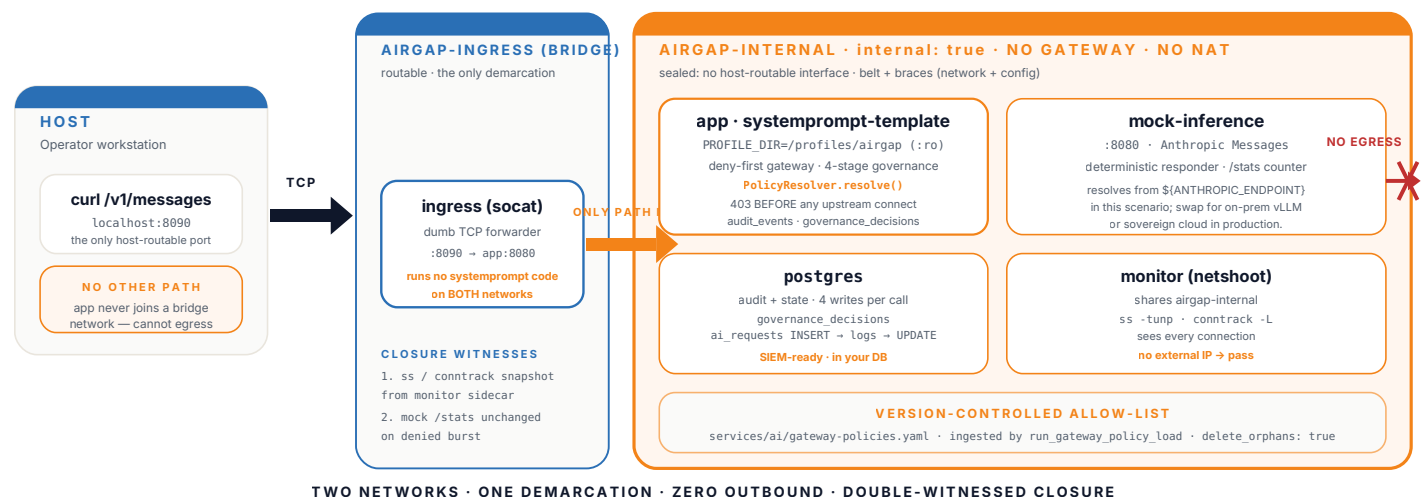
Evaluate the air-gap scenario free. **Sealed network, runnable on a laptop.**

`just airgap-up` brings up the sealed compose. `airgap-test` runs three assertion scripts: egress closure, sealed-network load, four-stage governance in isolation.

github.com/systempromptio/systemprompt-template

ed@systemprompt.io

03 Two networks, one demarcation: the sealed topology



ONE DEMARCATION The ingress socat is the only container on both networks; it forwards `:8090 → app:8080`. The app itself never joins a bridge — there is no host-routable interface for it to egress through, even if its config were misconfigured.

04 Sovereignty, on infrastructure you own

	Vendor SaaS	Cloud private	Air-gap + systemprompt.io
CLOSURE & RESIDENCY			
Outbound calls required	Yes · vendor cloud	Yes · cloud provider	None. <code>internal: true</code> · zero egress
Data residency	Vendor jurisdiction	Cloud region	Your datacenter. Your jurisdiction. ITAR / IL-pattern capable.
Deny-path proof	Vendor attestation	Cloud audit log	<code>/stats</code> counter unchanged · packet snapshot from sidecar
AUDIT & CONTROL			
Allow-list source-of-truth	Vendor console	Hand-seeded SQL / IaC	Version-controlled YAML · ingested every boot · <code>delete_orphans: true</code>
Audit ownership	Vendor-held	OpenTelemetry only	Your Postgres · 4 writes per call · joined by <code>trace_id</code>

05 Built for sovereign and defence deployments

FOR THE SOVEREIGNTY OFFICER

The proof is a packet snapshot.

No packet leaves the wall. Closure is asserted by a sidecar on the same sealed network — not by a vendor attestation, not by a cloud audit log. If the network ever opened, the next `01-egress-assert.sh` run would fail closed.

Outcome: Sovereignty you can re-prove on demand.

FOR THE AUDITOR

One SQL query, full lineage.

Every prompt → every policy decision → every tool call → every cost row, joined by `trace_id` to a JWT-verified identity. Run the query against your Postgres; the audit is structured evidence, not a policy document.

Outcome: Auditor asks, you run a SQL query.

Read the air-gap scenario walkthrough. Closure proofs, deny-first path, the four-stage governance pipeline.

Three runnable scripts: dual-layer egress assertion, sealed-network load + thresholds, four-stage governance in isolation. All artifacts under `results/`.

demo/scenarios/airgap/architecture.md